

近日，项目中有一个耗时较长的Job存在CPU占用过高的问题，经排查发现，主要时间消耗在往MyBatis中批量插入数据。mapper configuration是用foreach循环做的，差不多是这样。（由于项目保密，以下代码均为自己手写的demo代码）

```
1 <insert id="batchInsert" parameterType="java.util.List">
2     insert into table1 (field1, field2) values
3     <foreach collection="list" item="t" index="index" separator=",">
4         ({t.field1}, {t.field2})
5     </foreach>
6 </insert>
```

这个方法提升批量插入速度的原理是，将传统的：

```
1 INSERT INTO `table1` (`field1`, `field2`) VALUES ("data1", "data2");
2 INSERT INTO `table1` (`field1`, `field2`) VALUES ("data1", "data2");
3 INSERT INTO `table1` (`field1`, `field2`) VALUES ("data1", "data2");
4 INSERT INTO `table1` (`field1`, `field2`) VALUES ("data1", "data2");
5 INSERT INTO `table1` (`field1`, `field2`) VALUES ("data1", "data2");
```

转化为：

```
1 INSERT INTO `table1` (`field1`, `field2`)
2 VALUES ("data1", "data2"),
3 ("data1", "data2"),
4 ("data1", "data2"),
5 ("data1", "data2"),
6 ("data1", "data2");
```

虽然这个foreach看起来没有问题，但是经过项目实践发现，当表的列数较多（20+），以及一次性插入的行数较多（5000+）时，整个插入的耗时十分漫长，达到了14分钟，这是不能忍的。在资料中也提到了一句话：

```
1 Of course don't combine ALL of them, if the amount is HUGE. Say you have 1000 rows
  you need to insert, then don't do it one at a time.
```

- 2 You shouldn't equally try to have all 1000 rows in a single query. Instead break it into smaller sizes.

它强调，当插入数量很多时，不能一次性全放在一条语句里。可是为什么不能放在同一条语句里呢？这条语句为什么会耗时这么久呢？我查阅了资料发现：

- 1 Insert inside Mybatis foreach is not batch, this is a single (could become giant) SQL statement and that brings drawbacks:
- 2 some database such as Oracle here does not support.
- 3 in relevant cases: there will be a large number of records to insert and the database configured limit (by default around 2000 parameters per statement) will
- 4 be hit, and eventually possibly DB stack error if the statement itself become too large.
- 5
- 6
- 7 Iteration over the collection must not be done in the mybatis XML. Just execute a simple Insert statement in a Java Foreach loop.
- 8 The most important thing is the session Executor type.
- 9
- 10 Unlike default ExecutorType.SIMPLE, the statement will be prepared once and executed for each record to insert.

从资料中可知，默认执行器类型为Simple，会为每个语句创建一个新的预处理语句，也就是创建一个PreparedStatement对象。在我们的项目中，会不停地使用批量插入这个方法，而因为MyBatis对于含有<foreach>的语句，无法采用缓存，那么在每次调用方法时，都会重新解析sql语句。

- 1 Internally, it still generates the same single insert statement with many placeholders as the JDBC code above.
- 2 MyBatis has an ability to cache PreparedStatement, but this statement cannot be cached because it contains <foreach />
- 3 element and the statement varies depending on the parameters. As a result, MyBatis has to 1) evaluate the foreach part and 2)
- 4 parse the statement string to build parameter mapping [1] on every execution of this statement.
- 5 And these steps are relatively costly process when the statement string is big and contains many placeholders.
- 6 [1] simply put, it is a mapping between placeholders and the parameters.

根据上述资料可以看出，耗时主要花费在循环操作上。由于我使用foreach循环后有5000+个values，因此这个PreparedStatement特别长，包含了很多占位符，而占位符和参数的映射耗费了大量时间。此外，根据相关资料的研究表明，values的增加与所需解析时间呈指数型增长，因此这个循环操作需要更多的时间来完成。

foreach 遇到数量大，性能瓶颈

项目实践发现，当表的列数较多（超过20），以及一次性插入的行数较多（上万条）时，插入性能非常差，通常需要20分钟以上

这个时候就需要观察曲线了，10-100个来讲是很快的，当然也要根据项目请来看，总之建议100个就ok了，不要太高。

BatchInsert模式

也是官方针对批量数据插入优化的方法之一

<http://www.mybatis.org/mybatis-dynamic-sql/docs/insert.html>

```
1 SqlSession session = sqlSessionFactory.openSession(ExecutorType.BATCH);
2 try {
3     TableMapper mapper = session.getMapper(TableMapper.class);
4     //自定义你的方法来获取需要插入的数据
5     List<TableRecord> records = getRecordsToInsert();
6     //BatchInsert
7     BatchInsert<TableRecord> batchInsert = insert(records)
8         .into(table)
9         .map(id).toProperty("id")
10        .map(field1).toProperty("field1")
11        .map(field2).toProperty("field2")
12        .build()
13        .render(RenderingStrategy.MYBATIS3);
14    batchInsert.insertStatements().stream().forEach(mapper::insert);
15    session.commit();
16 } finally {
17     session.close();
18 }
```

即基本思想是将 MyBatis session 的 executor type 设为 Batch ，然后多次执行插入语句。就类似于JDBC的下面语句一样。

```
1 int insertNum = 100;
2 Connection connection = DriverManager.getConnection("jdbc:mysql://127.0.0.1:3306/xxx?
  useUnicode=true&characterEncoding=UTF-
  8&useServerPrepStmts=false&rewriteBatchedStatements=true","root","root123");
3 connection.setAutoCommit(false);
4 PreparedStatement ps = connection.prepareStatement(
5     "insert into table1(field1) values(?)");
6 for (int i = 0; i < insertNum; i++) {
7     ps.setString(1,"大狼狗"+insertNum);
8     ps.addBatch();
9 }
10 ps.executeBatch();
11 connection.commit();
12 connection.close();
```

经过实验，使用 ExecutorType.BATCH 的情况下，不到 2 秒就能全部插入完成，性能显著提升。

总结一下，如果使用Mybatis需要进行批量插入，推荐使用ExecutorType.BATCH的插入方式，如果非要使用<foreach>的插入的话，需要将每次插入的记录控制在20-50左右。

<https://wenku.baidu.com/view/2459326b132de2bd960590c69ec3d5bbfd0adaa2.html?wkt=1712136302497&bdQuery=ps.executeBatch%28%29%E5%8E%9F%E7%90%86&needWelcomeRecommend=1>